

UART + FIFO on Real FPGA

8-Week Execution Plan · Aug 1 – Sep 25, 2025

BITS Pilani Goa · EEE · 2nd Year · Continuation of 75-Day Sprint

8 weeks

compressed plan

No major exams

Aug–Sep window

3 project versions

sim → hw → AXI

SIP-ready

by end of Sep

WHAT YOU WILL HAVE BY SEP 25

- v1.0 — simulation: UART+FIFO fully verified in Icarus Verilog (done from 75 days)
- v2.0 — hardware: bitstream on Basys3/Nexys, loopback verified in PuTTY, demo video
- v3.0 — AXI: UART+FIFO wrapped in AXI4-Lite slave, register map, Vivado IP packaged
- Timing analysis: critical path identified, pipelined, max frequency documented
- SystemVerilog rewrite: always_ff/always_comb, assertions, cover properties
- GitHub tagged with 3 versions — tells a clear progression story in SIP interviews

Phase 1: Vivado + Constraints	Phase 2: Hardware Verification	Phase 3: Timing + SV	Phase 4: AXI4-Lite
Weeks 1–2	Weeks 3–4	Weeks 5–6	Weeks 7–8

Phase 1 — Vivado Setup & Constraints

Weeks 1–2 · Aug 1–14

Week 1

Vivado Setup + FPGA Flow Understanding

Aug 1–7

GOALS

- Install Vivado ML Edition and verify it opens without error
- Understand the Vivado RTL-to-bitstream flow: synthesis → implementation → bitstream
- Copy your UART+FIFO simulation project into a new Vivado project
- Run synthesis on your existing UART TX module — read the utilization report

TOPICS TO LEARN

Vivado	Project creation, source file import, simulation settings, synthesis run
RTL flow	Synthesis → Implementation → Bitstream — what each stage does and why
Reports	Utilization report: LUTs, FFs, DSP48s. Timing summary: WNS, TNS, critical path

DAILY SCHEDULE

MON	Install Vivado ML Edition if not done. Open it. Create a new RTL project. Import your and_gate.v as a test. Run synthesis. Confirm the tool works. This day is setup only.
TUE	Create a proper Vivado project. Import uart_tx.v, baud_gen.v, fifo.v, uart_fifo_top.v. Set top module. Run behavioral simulation in Vivado — verify waveforms match your Icarus results.
WED	Run synthesis on uart_fifo_top. Open Utilization Report — count LUTs and FFs used. Open Timing Summary — find WNS (worst negative slack). Write down both numbers.
THU	Fix any synthesis warnings (undriven outputs, multi-driven nets). Most warnings from simulation-only code (\$display, \$finish) — these must be removed from design files, not testbenches.
FRI	Bug drill: add a latch by removing else from a combinational always block in your FSM. Synthesize. Vivado will warn 'inferred latch'. See it in the schematic viewer. Fix it. This is a real synthesis issue.
SAT	Run implementation (place and route) after clean synthesis. Open timing report post-implementation. Note if WNS is positive (timing met) or negative (violation). If violation, note which path.
SUN	Review week. Document in README: what Vivado project structure looks like, what the utilization numbers were. Push to GitHub. Read ahead: what is an XDC constraints file?

DELIVERABLES — COMPLETE BY SUNDAY

■ Vivado project created with all UART+FIFO source files imported
■ Behavioral simulation in Vivado matches Icarus Verilog output
■ Synthesis completed — utilization report numbers noted
■ Implementation completed — timing report showing WNS
■ No critical synthesis errors (warnings acceptable at this stage)

CHECKPOINT Open the Utilization Report. How many LUTs does your uart_fifo_top use? If you cannot find this number in Vivado, you have not learned the tool yet.

BUG DRILL Latch inference from missing else in combinational always block. Vivado flags it explicitly — practice finding and fixing it.

Week 2

XDC Constraints + Board Pinout

Aug 8–14

GOALS

- Understand what an XDC file does and why it is required for hardware
- Write a complete XDC file for Basys3 or Nexys A7 board
- Map your UART TX pin to an actual PMOD or USB-UART pin on the board
- Understand clock constraint syntax and apply it to your 100 MHz board clock

TOPICS TO LEARN

XDC	set_property PACKAGE_PIN, IOSTANDARD LVCMOS33, clock constraints with create_clock
Board	Basys3: 100 MHz clock on W5, USB-UART TX on A18. Nexys A7: 100 MHz on E3, UART TX on D4
I/O	LVCMOS33 voltage standard, drive strength, slew rate — why they matter for signal integrity

DAILY SCHEDULE

MON	Download the official XDC file for your board (Digilent GitHub). Open it. Find: (1) the 100 MHz clock pin, (2) the USB-UART TX/RX pins, (3) the LED and switch pins. Mark them.
-----	---

TUE	Create your own minimal XDC for the UART+FIFO project: clock constraint, UART TX pin, 4 LED outputs for status flags (fifo_full, fifo_empty, tx_done, uart_busy). No copy-paste — type every line and understand it.
WED	Add clock constraint: create_clock -period 10.000 [get_ports clk]. Re-run implementation. Check if timing is now properly constrained. WNS should be positive for your design at 100 MHz.
THU	Add switch inputs to your top module: sw[7:0] as data input to FIFO write port. Button as wr_en pulse. This makes the design interactive on hardware.
FRI	Bug drill: set wrong IOSTANDARD (LVCMOS18 instead of LVCMOS33 for Basys3). Vivado will give DRC error. Read the error, understand what it means, fix it. This exact error happens to everyone first time.
SAT	Full clean run: synthesis → implementation → bitstream with your XDC. No errors. Green checkmarks on all three steps. This is your first real FPGA bitstream.
SUN	Review. You now have a bitstream file (.bit). Document the XDC file in your README with a comment explaining each pin mapping. Push to GitHub.

DELIVERABLES — COMPLETE BY SUNDAY

■ Custom XDC file with clock, UART TX, LED, and switch constraints
■ Clean implementation run with no DRC errors
■ Bitstream file (.bit) generated successfully
■ Top module modified: switches as data input, button as write enable
■ README updated with pin mapping table

CHECKPOINT Open your XDC file. Point to the line that constrains the clock. What is the period in nanoseconds? What frequency does that correspond to? If you cannot answer both, re-read the XDC.

BUG DRILL Wrong IOSTANDARD in XDC (LVCMOS18 vs LVCMOS33). Causes DRC critical error. Must be able to find and fix this independently.

Phase 2 — Hardware Verification

Weeks 3–4 · Aug 15–28

Week 3

Flash to Board + First Hardware Verification

Aug 15–21

GOALS

- Flash your bitstream to the FPGA board using Vivado Hardware Manager
- Verify the design is running by observing LED status signals
- Send a byte via UART and capture it on a serial terminal (PuTTY/TeraTerm)
- This is the week your simulation becomes physical hardware

TOPICS TO LEARN

Hardware Manager	Open target, auto-connect, program device — the Vivado flashing flow
Serial terminal	PuTTY or TeraTerm: set COM port, 9600 baud, 8N1, no flow control
USB-UART	How the Basys3/Nexys USB-UART bridge works — no external adapter needed

DAILY SCHEDULE

MON	Connect board to PC via USB. Open Vivado Hardware Manager. Open Target → Auto Connect. Program Device with your .bit file. LEDs should light up in a predictable pattern — confirms design is loaded.
TUE	Open Device Manager on Windows. Find the COM port for your board (Silicon Labs CP210x or FTDI). Open PuTTY: Serial, correct COM port, 9600 baud, 8-N-1. Leave terminal open.
WED	Press a switch combination on the board to write a byte into the FIFO. Press the button (wr_en). Watch the PuTTY terminal. You should see a character appear. If 0x41 was sent, 'A' should appear in PuTTY.
THU	Write multiple bytes by toggling switches and pressing button repeatedly. Verify all bytes appear in correct order in PuTTY — confirms FIFO is functioning on real hardware.
FRI	Debug session: deliberately write 9 bytes to 8-deep FIFO (overflow). Observe LED for fifo_full. Confirm dropped byte does not appear in PuTTY. Physical overflow handling verified.
SAT	Stress test: write bytes as fast as you can (human speed). Verify no data corruption. Send all 26 letters A-Z (0x41-0x5A) using switch combinations. Verify all appear in terminal.
SUN	Photograph or screen-record the PuTTY terminal showing received bytes. This is your demo evidence. Push all code and a photo to GitHub /docs folder.

DELIVERABLES — COMPLETE BY SUNDAY

■ Bitstream successfully flashed — board LEDs respond
■ PuTTY/TeraTerm configured and receiving bytes
■ Single byte transmission verified on hardware
■ Multi-byte FIFO drain verified on hardware
■ Photo/screenshot of terminal output saved in /docs

CHECKPOINT Type the hex value of ASCII 'B' (0x42) into your switch inputs, trigger send, and confirm 'B' appears in PuTTY. If it does, your UART TX is physically working at correct baud rate.

BUG DRILL If nothing appears in PuTTY: check COM port number, check baud rate is exactly 9600, check your baud divisor is 5208 for 50 MHz (Basys3 clock is actually 100 MHz — divisor should be 10416 at 100 MHz). This is a common first hardware bug.

Week 4

UART RX on Hardware + Loopback Test

Aug 22–28

GOALS

- Add UART RX to your hardware design
- Wire TX → RX internally for hardware loopback (no external cable needed)
- Verify: bytes sent via switches are received and displayed on LEDs/7-segment
- Display received byte on 7-segment display — visual hardware proof

TOPICS TO LEARN

UART RX	Your simulation RX module from Week 9 of the 75-day plan — now on hardware
Loopback	Internal loopback: connect uart_tx.tx port directly to uart_rx.rx port in top module
7-segment	Basys3 has a 4-digit 7-segment display. Drive it with received byte in hex — two digits

DAILY SCHEDULE

MON	Add uart_rx.v to your Vivado project. Wire internal loopback in top module: assign rx_in = tx_out. Re-synthesize and implement. Flash updated bitstream.
-----	--

TUE	Test loopback: send byte 0x41. Observe rx_done pulse on LED. Observe rx_data on 8 LEDs. Compare to what you sent — must match exactly.
WED	Add 7-segment display driver (write a simple hex_to_7seg decoder — you built this in Week 2 of the 75 days). Display lower nibble and upper nibble of received byte on 2 digits.
THU	Send bytes 0x00 through 0x0F one by one. Verify 7-segment shows 0,1,2...F. Increment through all 16 values. This confirms both TX and RX and display are all working.
FRI	Record a screen capture or phone video of the hardware demo: switch input → button press → 7-segment updates with received value. This is your demo video for GitHub.
SAT	Timing analysis: check that your RX 16x oversampling clock is correct for 100 MHz board clock. Divisor should be $100,000,000 / (9600 \times 16) = 651$. Verify in simulation, then on hardware.
SUN	Complete hardware verification. Update GitHub with demo video link, updated block diagram showing TX+RX+loopback, and updated README. Tag this commit as v2.0-hardware.

DELIVERABLES — COMPLETE BY SUNDAY

■ UART RX module added to hardware design
■ Internal loopback working — sent byte matches received byte on LEDs
■ 7-segment display showing received byte in hex
■ Demo video recorded and linked in README
■ GitHub tagged v2.0-hardware

CHECKPOINT Send 0xA5 (binary 10100101) via switches. Verify LEDs show 10100101 pattern AND 7-segment shows 'A5'. Both must match simultaneously.

BUG DRILL RX sampling wrong bit. If received byte is consistently shifted (e.g., send 0x41, receive 0x20 which is $0x41 \gg 1$), your bit counter is off by 1. Check if you are sampling 8 or 9 bits.

Phase 3 — Timing Closure & SystemVerilog

Weeks 5–6 · Aug 29 – Sep 11

Week 5

Timing Closure + STA Deep Dive

Aug 29 – Sep 4

GOALS

- Understand timing reports fully — not just green vs red
- Identify and understand the critical path in your UART+FIFO design
- Push clock to 150 MHz and observe timing failure — then fix it
- Learn the pipeline technique to fix a failing timing path

TOPICS TO LEARN

STA	Setup time equation: $\text{Slack} = T_{\text{clk}} - (T_{\text{cq}} + T_{\text{logic}} + T_{\text{net}} + T_{\text{setup}})$. Every term must be understood.
Critical path	The longest combinational path from any FF output to any FF input. Determines max frequency.
Pipelining	Break a long combinational path with an intermediate register — increases latency by 1 cycle, improves frequency.

DAILY SCHEDULE

MON	Open Timing Summary in Vivado after your Week 4 implementation. Find WNS. Click on it — Vivado shows the critical path. Identify: startpoint FF, combinational logic cells, endpoint FF.
------------	--

TUE	Change clock constraint to 150 MHz (period 6.667 ns). Re-run implementation. Check timing — likely fails. Note which path fails and by how much. Screenshot the failing report.
WED	Fix the failing path by pipelining it: find the long combinational path, insert a register stage in the middle. This adds 1 cycle latency but allows higher frequency.
THU	Re-run implementation at 150 MHz with pipelined design. Check if timing closes. If yes — you just improved your design's max frequency through pipelining. Document the before/after WNS.
FRI	Bug drill: set clock to 500 MHz. Run implementation. Vivado should show many violations. Read the report and identify how many paths fail and by how much total (TNS = total negative slack).
SAT	Return to 100 MHz. Flash final pipelined design. Verify loopback still works correctly at 100 MHz — pipelining should not have broken functionality, only added 1-2 cycle latency.
SUN	Document: what was the original max frequency? What was it after pipelining? This is a real interview talking point — 'I improved my design from 80 MHz to 150 MHz through pipelining the FIFO read path'.

DELIVERABLES — COMPLETE BY SUNDAY

■ Critical path identified and documented with screenshot
■ Pipelined version of the bottleneck path implemented
■ Before/after timing comparison documented
■ Design verified still functional after pipelining
■ Max frequency of design determined and documented

CHECKPOINT Without opening Vivado: explain the setup time equation and why inserting a pipeline register improves timing. If you can explain both without notes, you understand STA. This will come up in every hardware interview.

BUG DRILL Clock at 500 MHz causes dozens of violations. TNS (total negative slack) tells you the total of all violations. Understanding how to read TNS is as important as WNS.

Week 6

SystemVerilog Migration + Assertions

Sep 5–11

GOALS

- Rewrite your UART TX module in SystemVerilog style
- Use `always_ff`, `always_comb`, logic type instead of `reg/wire`
- Add immediate assertions to catch illegal FSM states
- Add a cover property to verify the happy-path through the FSM

TOPICS TO LEARN

SystemVerilog	<code>always_ff</code> (sequential), <code>always_comb</code> (combinational), <code>always_latch</code> (latch — avoid). logic replaces both wire and reg.
Assertions	<code>assert</code> (immediate check), <code>property + assert property</code> (concurrent), <code>cover property</code> (verify a scenario happened)
Interfaces	interface keyword: bundle related signals into one port — cleaner top-level connectivity

DAILY SCHEDULE

MON	Rewrite <code>uart_tx.v</code> as <code>uart_tx.sv</code> . Change <code>reg</code> → logic, <code>wire</code> → logic. Change <code>always @(posedge clk)</code> → <code>always_ff @(posedge clk)</code> . Change <code>always @(*)</code> → <code>always_comb</code> . Verify synthesis still clean.
------------	--

TUE	Add immediate assertions inside <code>always_ff</code> : <code>assert(state inside {IDLE, START, DATA, STOP}) else \$error('Invalid FSM state at time %0t', \$time)</code> . Simulate — confirm no assertion fires.
WED	Add concurrent assertion for UART frame: property <code>p_start_bit: @(posedge baud_tick) (send -> ##1 (tx == 0))</code> . <code>assert property(p_start_bit)</code> . This checks start bit follows send signal.
THU	Add cover property: <code>cover property (@(posedge clk) (state == IDLE) ##1 (state == START) ##8 (state == DATA) ##1 (state == STOP))</code> . Run simulation and confirm cover is hit.
FRI	Rewrite the FIFO module in SystemVerilog. Use a struct for read/write pointer pair. Use interface for FIFO signals. Cleaner code that looks like professional RTL.
SAT	Synthesize the SV version. Compare utilization to the original Verilog version — should be identical. Confirm Vivado accepts your SystemVerilog without issues.
SUN	Update GitHub: new /sv_version folder with all SystemVerilog files. README section explaining what changed and why SystemVerilog is preferred in industry.

DELIVERABLES — COMPLETE BY SUNDAY

■ UART TX rewritten in SystemVerilog (uart_tx.sv)
■ FIFO rewritten in SystemVerilog with struct and interface
■ Immediate assertions added to FSM
■ At least one concurrent assertion and one cover property
■ SV version synthesizes cleanly — matching utilization to original

CHECKPOINT Introduce an illegal FSM state transition in simulation (force state to 4'hF). Does your assertion fire with the error message? If yes, your assertions are working.

BUG DRILL Latch inferred in `always_comb` if not all outputs assigned in every branch. `always_comb` is stricter than `always @(*)` in simulation — it catches this earlier.

Phase 4 — AXI4-Lite Integration

Weeks 7–8 · Sep 12–25

Week 7

AXI4-Lite Wrapper — Week 1

Sep 12–18

GOALS

- Understand the AXI4-Lite protocol — 5 channels, handshake mechanism
- Implement the write address and write data channels (AW + W)
- Implement the write response channel (B)
- Wrap your UART TX as an AXI4-Lite slave register

TOPICS TO LEARN

AXI4-Lite	5 channels: AW (write addr), W (write data), B (write resp), AR (read addr), R (read data). Each has VALID/READY handshake.
Handshake	Transfer occurs when both VALID and READY are high simultaneously. Master drives VALID, slave drives READY.
Register map	UART control register at 0x00: [0]=send, [8:1]=tx_data. Status register at 0x04: [0]=tx_busy, [1]=fifo_full, [2]=fifo_empty.

DAILY SCHEDULE

MON	AXI4-Lite theory: draw the 5-channel handshake diagram on paper. For write: AW handshake → W handshake → B handshake. Read ARM IHI0022 (AXI spec) sections 2.1 and 2.2 — 30 minutes only.
TUE	Create axi_uart_slave.sv. Define ports: all 5 AXI channels + clk + rst + connection to uart_tx instance. Write the register file: 2 registers, each 32 bits wide.
WED	Implement AW+W channels: when AWVALID and WVALID both high, capture address and data, assert internal wr_en. Generate AWREADY and WREADY pulses. This is the write path.
THU	Implement B channel: after write completes, assert BVALID with BRESP=2'b00 (OKAY). Hold until BREADY from master. This acknowledges the write.
FRI	Write a testbench for the AXI write path. Drive: assert AWVALID with addr=0x00, assert WVALID with data=0x00000141 (send=1, tx_data=0x41). Verify BVALID goes high. Monitor UART TX output.
SAT	Simulate full write transaction: AXI write to control register → UART TX sends byte → verify 'A' transmitted on tx line in waveform.
SUN	Review AXI write path. Push code. Read ahead: AR and R channels for reading status register.

DELIVERABLES — COMPLETE BY SUNDAY

■ AXI4-Lite slave module with register map defined
■ Write address channel (AW) implemented with handshake
■ Write data channel (W) implemented with handshake
■ Write response channel (B) implemented
■ Testbench: AXI write triggers UART TX verified in simulation

CHECKPOINT In your testbench waveform: identify the exact clock cycle when AWVALID and AWREADY are both high (write address accepted). Then find when BVALID goes high (write acknowledged). The gap between them is your write latency.

BUG DRILL Deadlock: if you only assert AWREADY when WVALID is already high (or vice versa), master hangs waiting. AW and W channels must be accepted independently. Classic AXI beginner mistake.

Week 8	AXI4-Lite Wrapper — Week 2 + Full System Demo	Sep 19–25
--------	---	-----------

GOALS

- Implement AXI4-Lite read channels (AR + R) for status register
- Integrate complete AXI slave with uart_fifo_top
- Synthesize the AXI-wrapped system in Vivado
- Create final project documentation and demo

TOPICS TO LEARN

AR+R channels	Read: AR handshake → R data valid. Slave drives RVALID + RDATA when ARVALID received. RREADY from master confirms receipt.
IP Integrator	Vivado block design: drag your AXI slave IP, connect to Zynq PS or AXI Interconnect. This is how real SoC designs work.
Documentation	IP core README: register map table, timing diagram, usage example. Industry standard.

DAILY SCHEDULE

MON	Implement AR channel: when ARVALID high, capture address. Assert ARREADY. Implement R channel: put status register data on RDATA, assert RVALID. Hold until RREADY.
TUE	Full AXI testbench: (1) Write 0x141 to control reg → UART sends 'A'. (2) Read status reg → verify RDATA[0]=tx_busy=0 after transmission. Both read and write paths tested.
WED	Package your AXI UART as a Vivado IP: right-click in Sources → Create and Package New IP. This is what real IP cores look like — it generates a component.xml describing your interfaces.
THU	Synthesize AXI-wrapped design. Check utilization — how many more LUTs vs the original non-AXI version? Note the overhead of the AXI logic.
FRI	Flash AXI version to FPGA if time permits. Alternatively, record a full simulation demo: show the waveform of an AXI write transaction triggering a UART transmission end-to-end.
SAT	Final documentation sprint: register map table, complete block diagram showing AXI slave + UART TX + FIFO + handshake controller, waveform screenshots for AXI write and read transactions.
SUN	GitHub final push. Tag v3.0-axi. Update top-level README to show the evolution: v1.0 simulation → v2.0 hardware → v3.0 AXI. This progression tells a great story in interviews.

DELIVERABLES — COMPLETE BY SUNDAY

■ Complete AXI4-Lite slave: all 5 channels working
■ Full AXI testbench: write and read transactions verified
■ Vivado IP packaging attempted (component.xml generated)
■ Synthesis utilization comparison: baseline vs AXI overhead
■ GitHub v3.0-axi tagged with full documentation

CHECKPOINT Explain AXI4-Lite to yourself out loud in 60 seconds: what are the 5 channels, what is a handshake, what happens on a write transaction. If you can do this without notes, you are interview-ready on AXI.

BUG DRILL RDATA not registered: if you put combinational logic directly on RDATA without a register, it may glitch during AR handshake. Register RDATA on the cycle after ARVALID is asserted.

How to talk about this project in SIP interviews

30-SECOND VERSION

I built a UART transmitter and synchronous FIFO system in Verilog from scratch — starting with simulation in Icarus Verilog and GTKWave, then deploying on a Basys3 FPGA using Vivado. I verified serial transmission in PuTTY at 9600 baud, performed timing analysis to identify the critical path, and then wrapped the entire system in an AXI4-Lite slave interface. The design progressed through three versions: simulation, hardware deployment, and AXI integration.

QUESTIONS YOU MUST ANSWER COLD — NO NOTES

Q: Why is your baud divisor 5208?

A: $50,000,000 / 9600 = 5208.33 \rightarrow 5208$. At 100 MHz board clock it is 10,416.

Q: What is the FIFO full condition?

A: $(wr_ptr + 1) \% DEPTH == rd_ptr$. Not $wr_ptr == rd_ptr$ because that is empty.

Q: What is setup time slack?

A: $\text{Slack} = T_{\text{clk}} - (T_{\text{cq}} + T_{\text{logic}} + T_{\text{net}} + T_{\text{setup}})$. Positive = timing met. Negative = violation.

Q: How did you fix a timing violation?

A: Pipelining: inserted an intermediate register to break the critical path into two shorter paths.

Q: What is the AXI handshake?

A: Transfer occurs when both VALID (driven by initiator) and READY (driven by receiver) are high simultaneously on the same clock edge.

Q: What does synthesis do?

A: Converts RTL (behavioral Verilog) into a gate-level netlist — maps your logic to LUTs, FFs, and DSP blocks available on the FPGA fabric.